Objective audio quality characterization: technical report

Dominique Fourer and Geoffroy Peeters

October 11, 2016

Abstract

In this report, we propose a new set of audio features to describe the quality of an audio signal. Audio quality is here considered as being modified by the chain of processes/effects applied to the individual instrument tracks to obtain the final mix of a musical piece. Thus, the quality also depends on the mastering processes applied to the final mix or the signal degradation caused by MP3 compression. To evaluate our proposal, we created a large set of artificial mixes and also used real-world studio mixes. Using unsupervised and supervised classification methods, we show that our proposed audio features can detect the processing chain. Since this processing chain applied in professional studio has evolved over the years, we use our audio features to directly predict the decade during which a music track was recorded. Finally, we present a font-end tagging application designed for music collection description.

Contents

1 Introduction **2**
1.1 Notations . . . 3
1.1.1 Operators . . . 3
1.1.2 Variables . . . 3

2 The descriptors **3**
2.1 Compression and limiting . . . 4
2.1.1 Dynamic histogram . . . 4
2.1.2 Average spectrum . . . 4
2.1.3 Dynamic range compressor/expander parameters estimation . 5
2.2 Amplitude panning and stereophony detection . . . 6
2.2.1 Cochleagram differences . . . 6
2.2.2 Spectral Stereo Phase Spread . . . 7
2.2.3 DC-Offset . . . 8
2.2.4 Monophonic mixture detection . . . 8
2.2.5 Cross channel correlation . . . 9
2.2.6 Relative Delay . . . 9

2.2.7 Balance . . . 9
2.3 Signal content and degradations . . . 9
2.3.1 Root Mean Square (RMS) amplitude . . . 9
2.3.2 Spectral entropy . . . 10
2.3.3 Frequency bandwidth . . . 10
2.3.4 Background noise level . . . 10
2.4 Source separation . . . 11
2.4.1 Spatial diversity . . . 11
2.4.2 Synchronicity diversity . . . 11

3 Numerical experiments 11
3.1 Simulated altered signals . . . 11
3.1.1 Detection of a resampled signal . . . 11
3.1.2 Background noise detection . . . 12
3.1.3 Stereo effects detection . . . 13
3.1.4 Lossy audio compression characterization . . . 14
3.1.5 Detection of a dynamic range compressed signal . . . 14
3.2 Application on a realistic databases . . . 16
3.3 Scenario 1: prediction of audio signal degradations . . . 16
3.3.1 Supervised classification . . . 16
3.3.2 Unsupervised classification . . . 17
3.4 Scenario 2: Decade prediction . . . 18

4 Audio quality-based tagging application for music: documentation 19
4.1 Training details . . . 19
4.2 Application requirements and installation . . . 20
4.3 Files description . . . 20

5 Conclusion and future works 25

A Appendix 25

1 Introduction

This paper describes state-of-the-art and propose new features which can be used for the objective characterization of audio signal quality. This field of research recently gained of interest [1–7] with the requirement of indexing large music database made of various files generated dedicated for various media (e.g. MP3, broadcast, radio, CD, etc.)

Despite a consistent definition of audio quality has not yet been offered but researchers agree to say that it is related to the combination of transformations applied to the audio signal since audio sources recording or synthesis to the final result after the mixing and the mastering steps [6]. Thus, audio signal quality characterization can be related to audio reverse engineering [8] or audio source separation [9–11] and can find applications in audio branding [12], automatic music playlist generation [13] and automatic music mixing [14].

This paper is organized as follows. The signal descriptors are described in Section 2 and evaluated in Section 3. Finally, our results are discussed with future works direction in Section 5.

1.1 Notations

1.1.1 Operators

$x(t)$	continuous-time signal
$x[n]$	discrete-time signal sampled such as $t = nT_s$ (T_s being the sampling period)
$F_x^h(t, \omega)$	Short-Term Fourier Transform (STFT) of signal x using analysis window h
$F_x^h[n, m]$	discrete STFT of x defined as $F_x(nT_s, \frac{2\pi m}{MT_s})$
$\text{SNR}(x, \hat{x})$	Signal-to-Noise Ratio (SNR) defined as $10 \log_{10} \left(\frac{\lvert x \rvert^2}{\lvert x - \hat{x} \rvert^2} \right)$

1.1.2 Variables

N	finite length of a discrete signal (sample index is denoted n)
M	the number of frequency bins for the discrete STFT
L	number of channels (channel index is denoted l)

2 The descriptors

Table 1 summaries the list of all the descriptors proposed in this document. The details of each descriptor are referred in the Section column and the numerical evaluation is provided in Section 3.

Table 1: Summary of the proposed signal features for audio signal quality characterization.

Descriptor name	Abbr.	Designation	Section	Reference
Dynamic histogram	DynHist		§2.1.1	[7]
average spectrum	avSpec	mixture dynamic range	§**??**	[6]
Compressor parameters			§2.1.3	**new proposed**
Cochleagram difference	CochDiff		§2.2.1	[7]
Spectral Stereo Phase Spread	SSPS		§2.2.2	[7]
DC-offset	DCOff		§2.2.3	[15]
Monophony detector	isMono	stereo quality	§2.2.4	[15]
Cross channel correlation	CCCor		§2.2.5	[15]
Relative delay	RDelay		§2.2.6	[15]
Balance	Bal		§2.2.7	[15]
Root Mean Squared amplitude	RMS	signal content	§2.3.1	[16]
Spectral Entropy	SE	signal content	§2.3.2	[17]
Frequency bandwidth	BW	signal content	§2.3.3	[15]
Background noise level	BNL	signal content	§2.3.4	[15]
Spatial diversity	SDiv	source separation	§2.4.1	**new proposed**
Spectral synchronicity diversity	SyncDiv	source separation	§2.4.2	**new proposed**

2.1 Compression and limiting

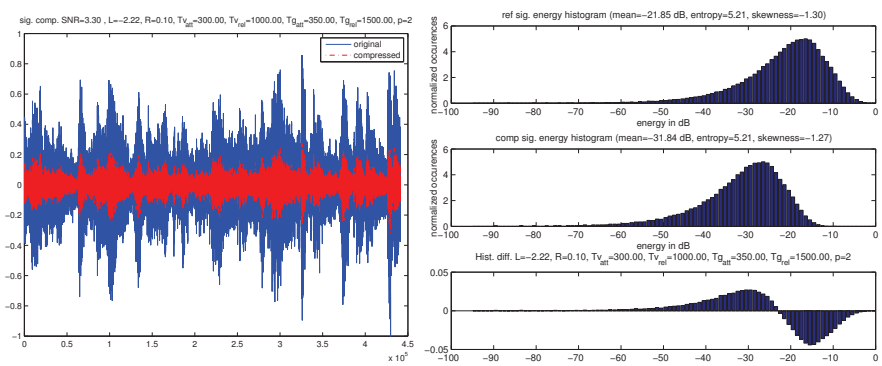
Compression and limiting effects are widely used for technical or artistic reason and aim at modifying the dynamic range of a musical piece. Thus, they can reveal information about the mixing process (*e.g.* the transformations applied on sound by sound engineers in studio). In [7, 18], the authors show relationship between the year of creation of a mastered mixture and the usage of a dynamic range compressor effect.

2.1.1 Dynamic histogram

It is computed from the normalized signal x expressed in dB as

$$x_{dB}[n] = 20\ \log_{10}\left(|x[n]|\right). \tag{1}$$

Hence, the histogram is computed from samples in the range $[-100, 0]$ dB with a resolution of 1dB [7] as illustrated in Fig. 1.



(a) signal (b) histogram comparison

Figure 1: Energy distribution of a musical piece as a function of the compressor parameters $[L, R, p, \tau_v^{\text{att}}, \tau_v^{\text{rel}}, \tau_g^{\text{att}}, \tau_g^{\text{rel}}]$ proposed in [19].

This distribution can also be summarized by computing statistics (*e.g.* mean, standard deviation, median or skewness).

2.1.2 Average spectrum

This descriptor introduced in [6] aims at describe the spectral envelope of a signal. It can be expressed as a function of the discrete STFT of a signal x as

$$\text{AS}[m] = \frac{1}{N}\sum_n |F_x[n,m]|^2 \ \text{ with } m \in [0, \frac{M}{2}[\tag{2}$$

2.1.3 Dynamic range compressor/expander parameters estimation

The parameters of the compressor model proposed in [19] can be estimated from the resulting signal in order to characterize the compression applied to a mixture. Furthermore, this model allows a signal reconstruction from the compressed signal when the parameters are known [8, 19].

Compressor description Let be x a 0-mean, normalized discrete-time input signal. The compressor parameters of model proposed by [19] are:

- L: the threshold expressed in dB
- R: the compression ratio expressed in $dB_{\text{in}} : dB_{\text{out}}$
- p: the compressor type (1:peak, 2:RMS)
- τ_v^{att}, τ_v^{rel} are the attack and the release time of the envelope expressed in seconds,
- τ_g^{att}, τ_g^{rel} are the attack and the release time of the gain filter expressed in seconds.

Hence, the compressed signal y is obtained by applying the following steps:

1. Envelope computation for attack and release detection:

$$v[n] = (\beta|x[n]|^p + (1-\beta)v[n-1]^p)^{1/p} \quad \text{with } \beta = 1 - \exp(-2.2/(F_s\tau_v)) \tag{3}$$

   τ_v^{att} is used instead of τ_v^{rel} when $|x[n]| > v[n-1]$.

2. Compression factor computation: $f[n] = \begin{cases} (l \cdot v[n]^{-1})^{1-\frac{1}{R}} & \text{when} \quad v[n] > l \\ 1 & \text{otherwise} \end{cases}$

   with $l = 10^{L/20}$

3. Smooth the compression factor as $g[n] = \gamma f[n] - (1-\gamma)g[n-1]$, with $\gamma = 1 - \exp(-2.2/(F_s\tau_g))$. τ_g^{att} is used instead of τ_g^{rel} when $f[n] < g[n-1]$)

4. Apply compression simply by $y[n] = g[n] \cdot x[n]$

Compressor parameters estimation In order to estimate the parameters of any compressor, x and y are supposed to be known and can be user both to obtain the following expressions.

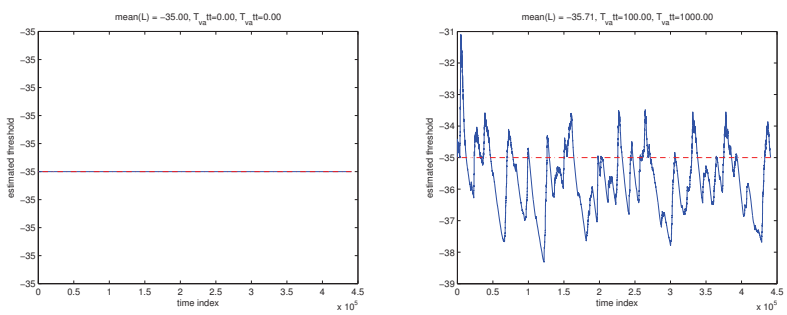
When $\gamma = 1$, we have:

$$y[n] = \begin{cases} x[n]\left(l \cdot v[n]^{-1}\right)^{\frac{R-1}{R}} & \text{when } v[n] > l \\ x[n] & \text{otherwise} \end{cases} \tag{4}$$

with $l = 10^{20/L}$ (constant) and $v[n]$ the envelope. Thus, we have

$$\hat{L}_R[n] = 20\log_{10}\left(v[n]\cdot\left(\frac{y[n]}{x[n]}\right)^{-\frac{R}{R-1}}\right) \tag{5}$$

Figure 2 shows the result provided by Eq.(5) when the assumption $\gamma = 1$ is or is not verified.



(a) $\gamma = 1$ ($\tau_g = 0$)

(b) $\gamma \neq 1$

Figure 2: Estimation of the threshold using Eq.(5) (x, y and R are assumed known). The exact parameter to estimate is $L = -35dB$

For the blind approach, the envelope can be approximated as a polynomial function directly from the compressed signal as proposed in [20].

According to the used model, the envelope $v[n]$, also can be expressed as a function of $x[n]$ and the parameters τ_v^{att}, τ_v^{rel} using Eq.(3). $v[n]$ verifies that if $\beta = 1$, we obtain a simple relation $v[n] = |x[n]|$, otherwise, if $\beta = 0$, $v[n] = |x[0]|$ is constant.

Thus, β can be estimated using the following relation obtained starting from Eq.(3):

$$\beta[n] = \frac{v[n]^p - v[n-1]^p}{|x[n]|^p - v[n-1]^p} \tag{6}$$

Thus τ_v in seconds is directly provided by $\tau_v = -\frac{2.2}{\log(1-\beta)F_s}$.

2.2 Amplitude panning and stereophony detection

This audio effect is used to distribute the energy of each source between the different channels. It is a common studio effect used to simulate the relative position of a sound source from the listener.

2.2.1 Cochleagram differences

In [7], the authors propose to use the difference between the Cochleagrams [21] computed for each channel as an indicator of the usage of the stereo panning [22]. The

Cochleagram of a signal x is a time-frequency representation obtained from the application of a gammatone filter bank on a signal expressed as

$$C(t, f_k) = \int_{\mathbb{R}} x(t-\tau) g_\lambda(\tau, f_k)\, d\tau \tag{7}$$

$$\text{with } g_\lambda(t, f) = \begin{cases} t^{n-1} \exp(-2\pi b_k t) \cos(2\pi f t) & \text{if } t > 0 \\ 0 & \text{otherwise} \end{cases} \tag{8}$$

where f_k and b_k denote respectively the central frequency and the width of band k, and λ denotes the order of filter g. As suggested by Patterson *et al.*in [23], we set $\lambda = 4$ that was shown to provide excellent fit to the human auditory filter shapes.

Thus, the Cochleagram can be defined as $|C(t, f_k)|^\alpha$ where α is a compression factor set equal to 1. In [7], the authors propose to use 70 filters with central frequencies taken from 30Hz to 11025Hz linearly spaced on the Equivalent Rectangular Bandwidth (ERB) scale approximated as [24]

$$\text{ERB}(f) \approx 21.4 \cdot \log_{10}\left(4.37 \cdot 10^{-3} f + 1\right). \tag{9}$$

From Eq.(7), the authors in [7] define the Cochleagram Difference (CD) as

$$\text{CD}(t, f_k) = |C_1(t, f_k)| - |C_2(t, f_k)| \tag{10}$$

where $C_1(t, f_k)$ and $C_2(t, f_k)$ are the resulting signals obtained after applying a gammatone filter bank respectively on the left and right channels signals, denoted $x_1(t)$ and $x_2(t)$. In our simulations, we used the implementation proposed in [21] for which Matlab code (mex) is freely available at `http://staffwww.dcs.shef.ac.uk/people/N.Ma/resources/ratemap`.

2.2.2 Spectral Stereo Phase Spread

This descriptor proposed by [7] measures the spread between both channels by computing the ratio of the standard deviations of the difference and of the addition of both channels. This can simply be computed for a 2-channels signal s as

$$\text{SPS} = \frac{\sigma(s_1 - s_2)}{\sigma(s_1 + s_2)} \tag{11}$$

where SPS stands for (stereo phase spread) temporal stereo phase $\sigma(x) = \left(\frac{1}{N-1}\sum_n (x - E(x))^2\right)^{1/2}$ is an unbiased estimator of the standard deviation of the time series x.

As proposed by the authors, this descriptor can also be computed in the frequency domain as

$$\text{SSPS}[m] = \frac{\sigma(S_1[n,m] - S_2[n,m])}{\sigma(S_1[n,m] + s_2[n,m])} \tag{12}$$

where S_1 and S_2 stand respectively for the STFT of s_1 and s_2.

A perceptual measure can be obtained by grouping frequencies into ERB bands as

$$\text{SSPS}[b] = \sum_{m_b} A[m]\text{SSPS}[m] \tag{13}$$

with $A[m] = \frac{1}{2N} \sum_{n=0}^{N-1} |S_1[n,m]| + |S_1[n,m]|$ the averaged modulus of the STFT for both channels at frequency m.

In our implementation we used the proposed ERB bands [24,25] (`https://en.wikipedia.org/wiki/Bark_scale`) detailed in Table 14.

2.2.3 DC-Offset

This descriptor proposed in [15] provides for each channel the average amplitude divided by the maximal absolute value of the amplitude for this channel, expressed as

$$\mathrm{DC}_l = \frac{\sum_{n=0}^{N-1} |x_l[n]|}{N \max(|x_l|)}, \tag{14}$$

for a L-channels signal x of length N.

2.2.4 Monophonic mixture detection

The standard ISO/IEC 15938 [15] suggests to use the cross channel correlation coefficient to detect if a mixture was originally recorded as mono. The whole algorithm can be expressed as

Algorithm 1 Detect if the L-channels signal x was originally mono ($L \geq 2$)

```
isOriginalMonoType ← false
for l ∈ [1, L] do
    x_l[n] ← x_l[n] / max(|x_l|)
    cxx_{1,l} ← xcorr(x_1, x_l)
    if l > 1 then
        cxx_{l,l} ← xcorr(x_l, x_l)
    end if
end for
cxx_max ← (cxx_{1,1}[0] + cxx_{L,L}[0]) / 2
for l ∈ [1, L] do
    cxx_{l,l} ← cxx_{l,l} / cxx_max
end for
if ∃i ∈ [−5, +5], ∃l ∈ [1, L], cxx_{l,l}[i] ≥ 0.99 then
    isOriginalMonoType ← true
end if
```

where $\mathrm{xcorr}(x, y)$ returns the cross correlation between vectors x and y computed as

$$\mathrm{xcorr}(x, y)[i] = \sum_{n \in \mathbb{Z}} x[n] \cdot y[n+i]. \tag{15}$$

2.2.5 Cross channel correlation

The cross channel correlation can simply be used as an informative feature to measure the redundancy between the first channel and the other $L-1$ channels [15]. The result is a vector of size $L-1$ computed as

$$r[l] = \frac{cxx_{1,l}[0]}{\sqrt{cxx_{1,1}[0]cxx_{l,l}[0]}} \quad l \in [1, L-1] \tag{16}$$

where $cxx_{l,m} \leftarrow \text{xcorr}(x_l, x_m)$ that can be computed using Eq.(15).

2.2.6 Relative Delay

This descriptor is proposed to compute the time delay between the first and the $L-1$ other channels [15]. It is computed as follows for a multichannel signal x

1. For each channel $l \in [2, L]$, compute the cross correlation $cxx_{1,l} = \text{xcorr}(s_1, s_l)$ computed as Eq.(15).
2. The relative delay of channel l is given by $\arg\max_n \left(cxx_{1,l}[n]\right)/F_s$
3. The confidence is given by the magnitude squared normalized cross correlation at maximum $n_{\max}$ expressed as

$$\text{MSC}_{l-1}[n_{\max}] = \frac{|cxx_{1,l}[n_{\max}]|^2}{cxx_{1,1}[n_{\max}]cxx_{l,l}[n_{\max}]}. \tag{17}$$

2.2.7 Balance

The balance descriptor corresponds to the averaged amplitude ratio between the first channel of a signal x and the $L-1$ other channels. This descriptor can be computed for each channel $l \in [2, L]$ as

$$B_l = 10\log_{10}\left(\frac{\sum_{n=0}^{N-1}|s_1[n]|^2}{\sum_{n=0}^{K-1}|s_l[n]|^2}\right) \tag{18}$$

2.3 Signal content and degradations

2.3.1 Root Mean Square (RMS) amplitude

This simple feature provides information about the content of the signal when it is applied on a zero-centered, single-channel signal mixture.

It can be defined for a signal x using a sliding window w of length $K < N$ as follows

$$\text{xRMS}[n] = \sqrt{\frac{1}{\sum_k w[k]}\sum_{k=-\lfloor K/2\rfloor}^{\lfloor K/2\rfloor}|x[n+k]\,w[k]|^2} \tag{19}$$

The common definition uses the entire signal with $w[n] = 1$ such that we have

$$\text{xRMS} = \sqrt{\frac{1}{N}\sum_{n=0}^{N-1} |x[n]|^2} \tag{20}$$

2.3.2 Spectral entropy

In [17], the authors propose to use the entropy as a feature to detect music in television audio recordings. More generally, the entropy is a measure of uncertainty or unpredictability of a random variable.

Thus, the time-frequency content of a single-channel signal x can be characterized by the Spectral Entropy (SE) descriptor defined as

$$\text{SE}[n] = -\sum_{m=0}^{M-1} P_x[n,m] \log_2 \left(P_x[n,m]\right) \tag{21}$$

where $P_x[n,m] = \frac{|F_x[n,m]|^2}{\sum_{m=0}^{M-1} |F_x[n,m]|^2}$ is the normalized instantaneous power spectrum that verifies $\sum_{m=0}^{M-1} P_x[n,m] = 1$.

2.3.3 Frequency bandwidth

This descriptor can help to detect signal resampling (artificial up-sampling) or signal source coding that is characterized by loss of information for high frequencies. The standard ISO/IEC 15938 [15] suggests to estimate this information from signal as follows

- The discrete spectrogram $|F_x^h[n,m]|^2$ of the analyzed signal is computed using 30ms-length frames with $1/3$-overlap. Each STFT is computed using $M = 2048$ discrete frequency bins.
- Each column vector of the spectrogram is normalized by the maximal value and converted to a logarithmic scale by $\text{LMPS}[n,m] = 10\log_{10}(|F_x^h[n,m]|^2)$
- The bandwidth is defined as the difference between the maximal and the minimal frequencies for which the energy of the STFT at instant n is above $0.7\max(\text{LMPS})$

2.3.4 Background noise level

The standard ISO/IEC 15938 [15] suggests to measure the **Background Noise Level (BNL)** of a signal x as fallows.

For each channel,

1. Compute the absolute maximal peak expressed in dB $\text{maxPeak}_{dB} = \max\left(|x[n]|\right)$
2. Divide x into 5ms blocks and find the block with the minimal mean energy expressed in dB minPow_{dB}
3. The background noise level is given by $\text{minPow}_{dB} - \text{maxPeak}_{dB}$

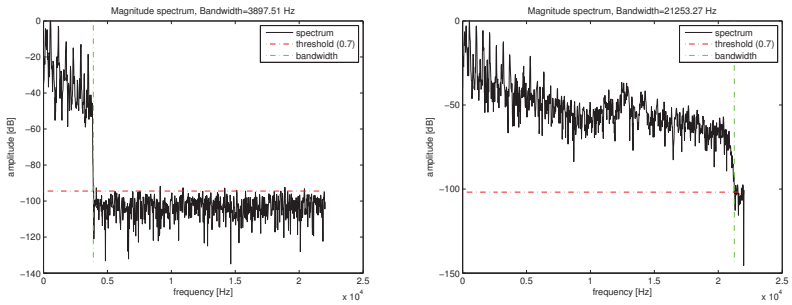


Figure 3: Instantaneous computation of the frequency bandwidth of a jazz signal mixture which was down-sampled from 44.1kHz to 8kH (on the left) and the reference (on the right).

2.4 Source separation

2.4.1 Spatial diversity

Here, we propose a new descriptor in order to describe how the audio sound sources present in a stereophonic audio mixture are distributed in different. [26]

2.4.2 Synchronicity diversity

TODO

3 Numerical experiments

3.1 Simulated altered signals

In this section, we propose to evaluate the proposed descriptors to detect alteration applied on signals.

3.1.1 Detection of a resampled signal

For this simulation, we consider a musical signal sampled at 44.1 kHz which is first resampled at different frequencies (8kHz, 16kHz and 22kHz) using sox. Second the downsampled signal is up-sampled to the original sampling frequency for this experiment.

Thus, the results provided by descriptor **Frequency Bandwidth (BW)** proposed in Section 2.3.3 are presented in Table 2. Mean, median, minimum and maximum values of the time series corresponding to the instantaneous estimated bandwidth computed each 30ms.

As expected, the results presented show that $\max(BW)$ provides a value close to $F_s/2$ that corresponds to the maximal frequency that can be represented according to Shannon theorem.

(a)	resampling at 8 kHz estimated BW (Hz)	mean 3922.75	median 3919.04	min 0	max 4543.51	std 105.69
(b)	resampling at 16 kHz estimated BW (Hz)	mean 7748.77	median 7795.02	min 3165.38	max 9883.74	std 393.11
(c)	resampling at 22.05 kHz estimated BW (Hz)	mean 10277.85	median 10615.87	min 0	max 11003.47	std 998.98
(d)	reference (44.1 kHz) estimated BW (Hz)	mean 12125.91	median 10788.13	min 0	max 21447.07	std 4962.29

Table 2: Estimated bandwidth of a musical signal mixture downsampled respectively at 8kHz (a), 16 kHz (b), 22.05 kHz and the reference at 44.1kHz(d).

3.1.2 Background noise detection

For this experiment, we observe the result provided by the **BNL** descriptor described in Section 2.3.4 for different input SNR resulting from the addition of a white Gaussian noise to a musical signal mixture.

SNR (dB)	-10	-5	0	5	10	15	20	25	30
BNL (dB)	-10.67	-13.45	-15.89	-17.40	-18.82	-19.31	-19.17	-19.42	-19.35
SE (bits)	9.23	8.91	8.34	7.65	7.03	6.53	6.14	5.88	5.73

Table 3: Background noise level (BNL) and spectral entropy (SE) descriptors as functions of the input signal SNR.

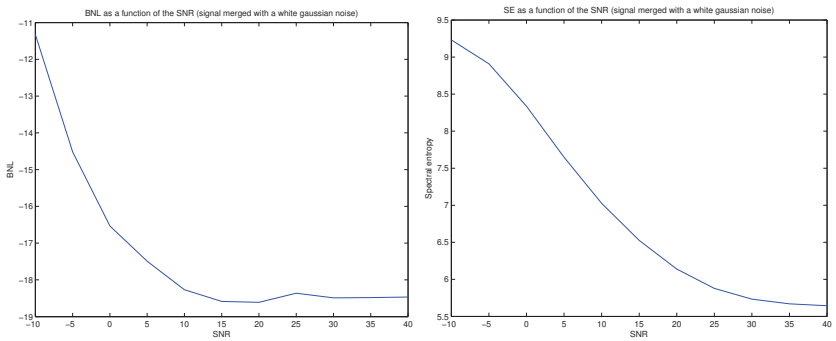


Figure 4: BNL and SE as a functions of the analyzed signal SNR.

The results presented in Table 3.1.2 show a relation between the input SNR and the results provided by BNL and SE descriptors. However for too high SNR, BNL looks less accurate and seems to clamp at a minimal value close to -20 dB as shown in Fig. 3.1.2.

3.1.3 Stereo effects detection

In this experiment we present the results provided by descriptors for different kind of altered stereo signals. Tables 4 shows the values obtained with simulated stereo alteration effects applied on a signal corresponding to a jazz musical piece .

	Bal	100· max(mean(CochDiff)	100· std(mean(CochDiff))	CCCor	DCOff ratio	SSPS	RD	isMono
original stereo	1.18	0.64	0.16	0.44	4.58	0.62	0	0
no stereo ($s_1 = s_2$)	1.00	0	0	1	1	0.02	0	**1**
opp. phases ($s_1 = -s_2$)	1.00	0	0	**-1**	-1	44.72	0	0
amplitude panning	29.94	0.8	0.4	1	1	0.53	0	1
phase panning	2.03	0.77	0.17	1	1	0.14	0	1
HRTF	2.21	1.81	0.99	0.13	2.01	0.89	0	0

Table 4: Averaged results obtained on simulated altered signals in presence of a white Gaussian noise on each channel (SNR $= 30$dB)

Rank	Descriptor name	score
1	isMono	1.97
2	100(std(CochDiff))	0.95
3	Relative Delay	0.08
4	SSPS	0.06
5	DCOff	0.01
6	100(max(CochDiff))	$6.3 \cdot 10^{-4}$
7	CCor	$4.3 \cdot 10^{-4}$
8	Bal	10^{-5}

Table 5: Ranking of the most relevant features to discriminate stereo effects, using the resulting LDA's eigenvectors as proposed in [27].

For this experiment, we consider a commercial stereophonic mixture that is manually altered by the application of classical stereophonic effects. For amplitude panning, phase panning, HRTF we use as parameter $\theta \in [-\frac{\pi}{2}, +\frac{\pi}{2}]$ which corresponds to the simulated direction from which the the audio signal is perceived. For this experiment, each resulting stereo mixture is merged with a white Gaussian noise to result a SNR equal to 30 dB. Each effect can be detailed as follows.

- **original stereo** is the original commercial mixture without transformations.
- **no stereo** corresponds to the mixture where the first channel is duplicated on the second channel, thus $s_1[n] = s_2[n]$
- **opposite phases** corresponds to the mixture where the first channel is the opposite of the second one, hence, $s_1[n] = -s_2[n]$
- **amplitude panning** is a simple Balance effect which consists in simulating a stereo mixture by changing the balance between both channels. For our experiment we have $s_1 = 0.65s$ and $s_2 = 0.35$,

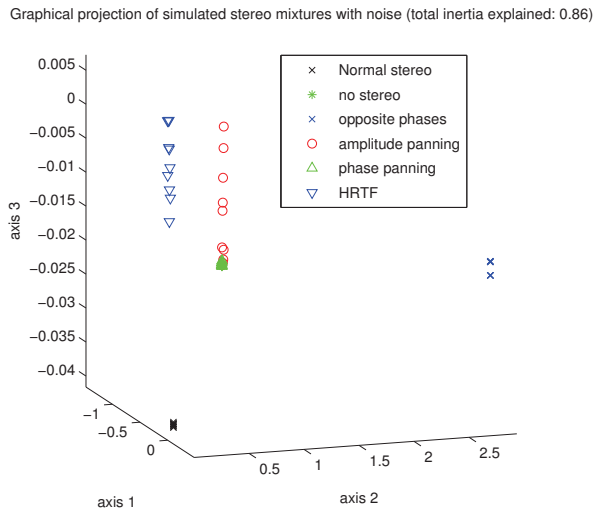


Figure 5: Result of the LDA applied on simulated stereo effects applied on the same input signal (original stereo).

- **phase panning** is a simple spatialization effect which add delay between both channels, hence we have $S_2(t,\omega) = S_1(t,\omega)\,\mathbf{e}^{j\Delta\Phi}$ where S_1 and S_2 are the STFT of s_1 and s_2.
- **HRTF**(Head Related Transfert Function)

3.1.4 Lossy audio compression characterization

For this experiment, we apply MPEG Audio Layer III (MP3) compression on a stereo mixture originally sampled at 44.4 kHz. The compression is obtained using the LAME[1] MP3 encoder which is licensed under LGPL and provide high quality audio signals.

3.1.5 Detection of a dynamic range compressed signal

For this experiment, we applied different sox[2] commonly used limiter/expander profiles summarized in table A.

Profile:	speech 1	speech 2	speech/streaming	streaming 1	streaming 2	speech/streaming	speech/music 1	speech/music 2	music 1	music 2	music 3	music 4	music 5
SNR(dB)	-11.94	-7.99	-4.19	2.91	-12.84	-4.59	16.69	0	-12.35	-11.40	-23.56	11.74	11.63

[1] `http://lame.sourceforge.net/`
[2] http://sox.sourceforge.net/

MP3 bitrate (kbs)	no compression	320	128	64	32	8
SNR(dB)	∞	32.36	17.48	13.57	-2.41	-2.41
BNL(dB)	-10.79	-10.72	-10.90	-10.43	-9.68	-11.20
SE(bits)	5.53	5.54	5.59	5.52	5.26	4.36
Bandwidth(kHz)	20.62	19.93	17.78	14.44	9.34	6.33
DynHist mean(dB)	-39.51	-39.50	-39.85	-39.78	-39.83	-40.05
DynHist std(dB)	12.70	12.69	12.66	12.64	12.62	12.51
DynHist skw(dB)	-0.71	-0.70	-0.70	-0.72	-0.71	-0.72
DynHist entropy(bits)	5.61	5.61	5.61	5.63	5.63	5.62

Table 6: Background Noise Level, Spectral entropy, frequency bandwidth and dynamic histogram of resulting MP3 compressed signals.

3.2 Application on a realistic databases

In order to validate our proposed audio quality features, we use the following two experimental scenarios.

3.3 Scenario 1: prediction of audio signal degradations

In this scenario, we used the Medley database [28] which provides 122 music pieces available in two forms: 1) studio mix (which is a stereo high-quality artistic mixture with effects applied to the various instrument tracks), 2) the set of separate multi-track instruments which allows to build a flat monophonic mix (named **li!** (**li!**) mix). On those, we apply a set of 27 different signal alteration effects. Those are detailed in Section **??** and summarized in Table 7. It should be noted that several sets of parameters can be used for a given effect to obtain several instances (*e.g.* dynamic range compression uses 5 different profiles).

Table 7: List of considered simulated audio alteration.

Effect name (# of classes)	Profiles	#
Dynamic range control (7)	no compression (**li!** mix)	1
	reference studio mix	1
	dynamic range compression (sox)	5
Spatialization (5)	reference studio mix	1
	mono	1
	amplitude panning	4
	phase panning	4
	HRTF	4
Lossy compression (5)	original WAV file	1
	MP3 compression (LAME encoder)	4
Content alteration (10)	resampling	5
	addition of a white Gaussian noise	5

The goal of our experiment is to automatically recognize the alteration effects applied to the audio, using our audio features (*cf.* Table 1). We consider this as a set of classification tasks with 7 classes for the dynamic range control, 5 for spatialization, 5 for lossy compression and 10 for content alteration. We try to solve these tasks using both supervised and unsupervised classification.

3.3.1 Supervised classification

We tested the following supervised classification algorithms: **knn!** (**knn!**), Linear Discriminant Analysis (LDA) [29] and multiclass (one-against-all) **rbf!** (**rbf!**) kernel Support Vector Machines (SVM) [30]. We performed a 3-folds cross-validation, with randomly partitioned equal sized folds. For the **knn!** method, we used $K = 9$ (which was empirically found to provide the best results) and a city-block distance (also named Manhattan) which provided better results than an Euclidean one. The classification results are indicated in Table 8 in terms of class-recall and global accuracy. The corresponding confusion matrices are indicated in Tables 9 (a)-(d).

The table shows that SVM and LDA outperform the **knn!** method in all cases. Best results are obtained to predict spatialization classes (98%), then lossy compression (88%), then content alteration (85%) and finally Dynamic Range (71%). It should be

Table 8: Supervised classification results for each task in terms of recall and of accuracy.

Method	Dynamic range control class name							Accuracy
	no co.	stud.	spee.	stream.	spe./mus.	mus.1	mus.2	
KNN	0.36	0.80	0.23	0.08	0.26	0.44	0.06	0.32
LDA	0.72	0.98	**0.65**	**0.48**	**0.89**	**0.96**	**0.27**	**0.71**
SVM	**0.90**	**0.99**	0.48	0.37	0.23	0.95	0.09	0.57

Method	Spatialization class name					Accuracy
	stud. mix	mono	amp. pan.	phs. pan.	HRTF	
KNN	0.31	0.34	0.90	0.85	0.98	0.83
LDA	0.94	**1**	0.97	0.57	**1**	0.86
SVM	**0.96**	0.89	**1**	**0.97**	0.99	**0.98**

Method	Lossy compression class name					Accuracy
	orig. wav	mp3 320kbs	mp3 128kbs	mp3 64kbs	mp3 16kbs	
KNN	0.34	0.20	0.20	0.99	**1**	0.55
LDA	0.73	**0.80**	**0.85**	**1**	**1**	**0.88**
SVM	**0.75**	0.59	0.43	**1**	0.99	0.75

Method	content alteration class name										Acc.
	8kHz	16kHz	32kHz	44kHz	96kHz	-15dB	-5dB	10dB	20dB	45 dB	
KNN	0.83	0.72	0.51	0.25	0.32	**1**	**1**	0.90	0.61	0.24	0.64
LDA	0.87	**0.89**	**0.81**	0.55	**0.68**	**1**	**1**	**98**	**0.94**	**0.77**	**0.85**
SVM	**0.90**	0.80	0.70	**0.57**	0.65	0.99	**1**	0.89	0.66	0.46	0.76

noted that discriminating the Dynamic Range classes is a much harder problem but interestingly, our method achieves (but with difficulties according to Table. 9 (a)) to discriminate the "streaming" and the "music 2" profiles which use almost identical SoX parameters.

Table 9: Confusion matrices obtained using the best classification method for each prediction task.

(a) LDA method applied to dynamic range control effect prediction.

ref. class	Classified as						
	no comp.	stud. mix	spee.	stream.	spe./mus.	mus.1	mus.2
no comp.	**88**				4	30	
stud. mix		**119**	1		1	1	
spee.			**79**	14	8		21
stream.			9	**59**	1		53
spe./mus.			4		**109**		
mus.1					5	**117**	
mus.2			40	**49**			33

(b) LDA method applied to dynamic range control effect prediction.

ref. class	Classified as									
	8kHz	16kHz	32kHz	44kHz	96kHz	-15dB	-5dB	10dB	20dB	45 dB
8kHz	**106**	13								3
16kHz	4	**119**		6	1	1				3
32kHz		2	**99**	5	14					2
44kHz	1	6	7	**67**	19					22
96kHz		2	15	19	**83**					3
-15dB						**122**				
-5dB							**122**			
10dB								**119**	3	
20dB								2	**115**	5
45dB		3		20					5	**94**

(c) LDA method applied to lossy compression detect.

ref. class	Classified as				
	wav	320kbs	128kbs	64kbs	16kbs
orig. wav	**89**	25	8		
mp3 320kbs	18	**97**	7		
mp3 128kbs	5	13	**104**		
mp3 64kbs				**122**	
mp3 16kbs					**122**

(d) SVM method applied to spatialization effect detect.

ref. class	Classified as				
	stud.mix	mono	amp. pan.	phs. pan.	HRTF
stud.mix	**117**			1	4
mono		**108**		14	
amp. pan.			**487**	1	
phs. pan.		2	11	**475**	
HRTF	1	2			**485**

(e) SVM method applied to decade predict.

ref. class	Classified as				
	60s	70s	80s	90s	2000s
60s	**327**	49	8	7	5
70s	73	**123**	**136**	45	19
80s	9	74	**272**	35	6
90s	26	29	61	**216**	64
2000s	11	11	16	44	**314**

3.3.2 Unsupervised classification

For the unsupervised classification, we use a simple k-means algorithm [31] using again the city-block distance. The number K of clusters is fitted to the number of classes to discriminate (*cf.* Table 7). For the unsupervised case, we used an automatic feature selection algorithm in order to detect the most relevant features to be used for each classification task. For this, we used the Inertia Ratio Maximisation using features space projection (IRMFSP) [32] algorithm. Thus, the most informative features are indicated in Table 11.

In Table 10 we indicate the results of the unsupervised classification in terms of cluster purity [33] (a cluster containing a single class has a purity of 1). Using less than 7 signal features, we obtain purities above 0.6 for all classification tasks. As for the supervised case, the spatialization classes are also the best predicted in the unsupervised case. These results are very promising for future unsupervised applications.

Table 10: Unsupervised classification results for each task in terms of cluster purity, optimal number of features, number of clusters.

Task	Cluster purity	# of feat.	K
Dynamic range control	0.62	4	7
Spatialization	0.80	5	5
Lossy compression	0.78	3	5
Resampling	0.71	2	5
Noise add.	0.78	7	5
Noise add.+Resampling	0.63	6	10

Table 11: Top-10 features sorted by descending order of the Fisher score (FS) [32] for each classification task.

rank	dynamic range control		spatialization		lossy compression		content alteration	
	feat. name	FS	feat. name	FS	feat. name	FS	feat. name	FS
1	aRMS	0.80	isMono	0.71	mean AS	1	median AS	1
2	SSPS	0.70	CCCor	0.60	slope AS	0.96	mean BW	0.74
3	min DH	0.42	CD5	0.53	max BW	0.39	max BW	0.69
4	CCCor	0.23	SSPS	0.45	mean BW	0.22	min SE	0.33
5	DH pk. pos.	0.13	CD1	0.23	median AS	0.06	mean SE	0.28
6	CD1	0.05	CD4	0.07	std BW	0.06	skew. SE	0.18
7	entropy DH	0.04	aRMS	0.05	std AS	0.05	median SE	0.16
8	skew. DH	0.03	CD3	0.04	max AS	0.02	max SE	0.14
9	std. DH	0.02	Bal	0.02	skew. BW	0.02	entropy SE	0.12
10	slope DH	0.01	slope AS	0.02	iqr BW	0.02	min DH	0.10

3.4 Scenario 2: Decade prediction

Since the processing chain applied in professional studio has evolved over the years, and since our audio features allows describing this chain, we test the use our audio features to predict directly the decades during which a music track was recorded. For this, we consider the dataset of 1980 music tracks previously used in [7]. It covers the years from 1960 to 2000. We subdivide them into 5 decades considered as classes. Each musical piece is resampled at 44.1 kHz and only the 60 first seconds are analyzed. As before, we tested the following supervised classification algorithms: **knn!**, LDA and SVM using a 3-folds cross-validation scheme. We also applied an artist filter in order to ensure that the same artist is not present both in the training and testing set (it allows to prevent over-fitting [34]).

Results are indicated in Table 12 in terms of recall and accuracy. The best results are obtained using SVM (63%). In comparison, the results obtained in [7] were 61% without adding **mfcc!** (**mfcc!**) to the set of audio quality features and 64% including them. It should be noted however, that the splitting between train and test used here is not the same as the one used in [7].

Table 12: Supervised classification results for decade prediction in terms of recall and of accuracy.

Method	Class name					**Accuracy**
	60s	70s	80s	90s	2000s	
KNN	0.77	0.38	0.63	0.49	0.71	0.60
LDA	0.69	**0.43**	0.62	0.52	0.77	0.60
SVM	**0.83**	0.31	**0.69**	**0.55**	**0.79**	**0.63**

4 Audio quality-based tagging application for music: documentation

A stand-alone application based on the features previously described is implemented for Matlab and available on the IRCAM repository at:
`/net/inavouable/u.anasynth/fourer/27_ICASSP17_IRCAM/mfiles`.
This application is made of 3 main components which respectively correspond to:

- `App_learnModels.m`: the training module which learn the modeles used during the tagging process based on supervised classification,
- `App_audioquality.m`: the main application which computes the tags from an input wav file
- `demo_AppAQ_tagging`: a demo-application which presents a music tagging application demonstration.

A classical usage case consists in training models based on the desired taxonomy. Then, 6 tags related to the audio quality are provided based on supervised or unsupervised classifcation to obtain the corresponding tags related to 1)dynamic range compression, 2)spatialization effects, 3)lossy compression bitrate, 4)content frequency bandwidth, 5) Noise level and 6)predicted decade of mixture creation.

4.1 Training details

`App_learnModels` is pre-required to compute first the audio-quality models stored in the following files.

- `App_model_dr.mat`: dynamic range compression model. The considered classes are:
  `SoX` dynamic compression profiles as detailed in [35] (`compression_tags`={'no compression', 'speech 1', 'streaming 1', 'speech/music 1', 'music 1'})
- `App_model_sp.mat`: spatialization model. (`spatialization_tags`={ 'mono', 'opposited phases', 'studio mix', 'amplitude panning', 'phase panning', 'HRTF'}.
- `App_model_lc.mat`: lossy compression model (`lossy_compression_tags`={'320 kbs', '128 kbs', '64 kbs', '16 kbs'}).

- `App_model_bw.mat`: frequency bandwidth model (`resampling_tags`={'8kHz', '16kHz', '32kHz', '44.1kHz'}).
- `App_model_dg.mat`: noise degradation model (`noise_tags`={'high SNR', 'medium SNR', 'poor SNR'}).
- `App_model_dec.mat`: decade prediction model (`decades_tags`={'60s', '70s', '80s', '90s', '2000s'}).

4.2 Application requirements and installation

This application is designed for MATLAB and uses the following external toolboxes.

- `SoX` should be present on the system where the application is used (*e.g.* `apt-get install sox` on a `.deb` package-based system).
- `lame` hould be present on the system where the application is used (*e.g.* `apt-get install lame` on a `.deb` package-based system).
- `MSVMpack` [30] is located in the `MSVMpack1.5` folder. It can simply be installed using `make` in a terminal and `addpath('MSVMpack1.5/matlab')` in Matlab.
- `Ratemap` [21] is located in the main folder and is used to compute cochleagrams (*cf.* Section 2.2.1). The installation is simply done by the command `mex makeRateMap_c.c` in Matlab.
- `mp3toolbox` is located in folder `mp3_toolbox` and is installed by `addpath('mp3_toolbox/mp3reader')` and `addpath('mp3_toolbox/mp3write')`. This toolbox ise used to handle the mp3 file format. To be used on a linux kernel system, `mpg123` should be installed (*e.g.* `apt-get install mpg123` on a `.deb` package-based system).
- `Ircam MFCC` is provided in `mfcc` sub-folder containing compiled binary for linux 32bits and 64bits.

4.3 Files description

The content of the repository is detailed in Table 13 which contains a short description of each Matlab function or script. A detailed usage description of each file can be obtained by `help function_name` in Matlab.

Table 13: Repository files description.

Filename	Description
afd.m	(function) Compute discriminant factor analysis from data (also called LDA).
alpha_spat.m	(function) Used to compute the α parameter in model-based HRTF spatialization.
amp_panning.m	(function) Apply the amplitude panning effect.
annotation2data.m	(function) Extract data (matrice of features) with class indices from a text file.
App_audioquality.m	(function) Compute audio quality tags from an input wav file.
App_learnModels.m	(script) Learn audio quality models used by App_audioquality.m
artist_collision_folds.m	(function) Compute the number of occurences for each artist in each fold.
artist_filtering.m	(function) Apply artist filtering to reduce the number of occurences of each artist in each fold. (each artist should appear in a unique fold)
artists_in_folds	(function) Compute the number of occurence of each artist in each fold.
average_spectrum.m	(function) Compute the average spectrum (AS) feature from input signal.
BackgroundNoiseLevel.m	(function) Compute the background noise level (BNL) feature from input signal.
Balance.m	(function) Compute the balance (Bal) feature from input signal.
Bandwidth.m	(function) Compute the bandwidth (BW) feature from input signal.
barkscale.m	(function) Compute the bark critical bands for a given number of discrete frequency bins.
beta_spat.m	(function) Used to compute the β parameter in model-based HRTF spatialization.
centred_data.m	(function) Center and reduce a matrix of features (data).
clean_data	(function) Remove invalid elements (Nan and Inf) from a matrix of features (data).
cochleaDiff.m	(function) Compute the cochlea difference (CD) feature from input signal.
compressorSG.m	(function) Apply model-based invertible dynamic range compression [19].
compressor_sox.m	(function) Apply SoX dynamic range compression.
compresstomp3.m	(function) Apply MP3 compression at user-defined bitrate to the input signal.
compSG_param.m	(deprecated function) Compute the dynamic range compression model parameters from the original signal and its compressed version.
create_LI_mix.m	(function) Create a linear instantaneous (LI) mix using the wav files contained in the target folder.
create_mixture.m	(function) Create a signal mixture using the wav files contained in the target folder and the given complex mixing matrix.
crossChannelCorr.m	(function) Compute the Cross channel correlation (CCCor) feature from input signal.

Filename	Description
DcOffset.m	(function) Compute the DC-offset (DCOff) feature from input signal.
demo_AppAQ_tagging.m	(script) Demo application of the audio quality tagging application.
demo_ratemap.m	(script) Demo application showing the usage of Ratemap for cochleagram computation.
disp_afd.m	(function) Display the projected data on the discriminant subspace provided by the LDA method.
dynamic_h	(function) Compute the dynamic histogram feature from input signal.
eps2pdf	(function) Convert all the eps present in the specified folder to pdf.
F_audioQ_desc.m	(deprecated function) compute 25 audio quality features from input signal.
F_audioQ_desc2.m	(function) Compute the 57 audio quality features detailed in [35] from input signal.
F_classif_nfold_valid.m	(deprecated function) Compute a n-fold cross validation classification evaluation.
F_classif_nfold_valid2.m	(function) Compute a n-fold cross validation classification evaluation.
Fig_rms.m	(script) Display the max amplitude RMS feature as a function of the window length.
F_soxresample.m	(function) Resample a input signal at a given sampling frequency using SoX.
F_unsupervised_class	(function) Apply unsupervised classification and return the cluster purity
F_unsupervised_range	(function) Apply unsupervised classification for a varying number of features, and return the best cluster purity.
fusion_data.m	(function) Merge 2 data matrices with their annotations indices.
gen_tab.m	(function) Return a string containing the provided table results to the latex format.
hist2dataW.m	(function) Convert a 2D histogram to a 3-dimensions tensor.
indice2group	(function) Convert a indices matrix (where each line contain the indices range of each label) to a vector containing the labels of each index.
inst2indices	(function) Convert a inst structure to a indices matrix.
inv_spectrogram.m	(function) Compute the inverse STFT from the input STFT matrix.
irmfsp.m	(function) Compute the IRMFSP algorithm [32].
isodd.m	(function) Return true if the input number is odd
isOriginalMono.m	(function) Compute the isMono feature from input signal.
kbdwin.m	(function) Compute the Kaiser-Bessel derived window.
label_index.m	(function) Return the index of a label in a given dictionary (and add entry in dictionary if not present).
Localize.m	(function) Apply binaural localization from the input 2-channels signal.
makeRateMap_c.m	(function) Ratemap toolbox [21] used to compute a cochleagram.
MatEntropy.m	(function) Compute the entropy of the input matrix.
mean_purety.m	(function) Compute the mean cluster purity from input data matrices and corresponding purity.

Filename	Description
`mfcc.m`	(function) Compute the MFCC (Dan Elis implementation).
`modulo2pi.m`	(function) Restrict a number in $[-\pi, +\pi]$ modulo 2π.
`M_prepare_database.m`	(script) Compute the audio quality descriptors for each file contained in a database and store the results in specified annotations folder.
`M_start_decade_classif.m`	(deprecated script) Decade prediction classification.
`M_start_decade_classif2.m`	(script) Decade prediction classification based on 2-folds, used-defined training / tests datasets.
`M_start_decade_classif3.m`	(script) Decade prediction classification based on random 3-folds validation, with artist filtering (used in paper [35]).
`M_start_degradation.m`	(script) Apply signal alteration on MedleyDB and compute AQ descriptors (used in paper [35]).
`M_start_degradation_classif.m`	(script) Signal alteration prediction based on random 3-folds validation (used in paper [35]).
`my_confusionmat.m`	(function) Compute a confusion matrix from 2 vectors of labels g and $\hat{g}$.
`my_filter.m`	(function) Filter a signal using provided coefficients a and b.
`my_fuzzWkmean`	(function) Apply the fuzzy k-mean algorithm.
`my_histogram`	(function) Compute and display the histogram of the input time series.
`my_mfcc.m`	(function) Compute the MFCC using the IRCAM MIREX MFCC binary implementation (located in mfcc sub-folder).
`my_normalize.m`	(function) Normalize input signal.
`parsermfcc.m`	(function) Convert to matlab format the MFCC computed by IRCAM MIREX MFCC.
`phase_panning.m`	(function) Apply the phase panning effect to the input signal.
`phase_stereo.m`	(function) Compute the SSPS descriptor (bark scale).
`phase_stereo2.m`	(function) Compute the SSPS descriptor with consideration to frequency (bark scale).
`pred_lda.m`	(function) LDA classification using the previously computed model (`train_lda`).
`rastaplp.m`	(functionn) Compute cepstral coefficient (Dan Ellis implementation).
`reduced_time_axis.m`	(function) Compute a time axis of the given length at the user-defined sampling frequency F_s.
`regression1d.m`	(function) Apply linear regression to obtain a slope and the origin from a set of points.
`RelativeDelay.m`	(function) Compute the RelativeDelay feature from input signal.
`rescale_vec.m`	(function) Force the size of the vector x to the target length.
`rms.m`	(function) Compute the amplitude RMS feature from input signal.
`sigmerge.m`	(function) Merge two signal together at a given ratio expressed in dB.
`SNR.m`	(function) Compute the SNR between two signal s and $\hat{s}$ (the noise corresponds to $s - \hat{s}$).
`SpatialDiversity.m`	(deprecated function) Compute the spatial diversity new feature from input signal (in development).
`spatialize_hrtf.m`	(function) Apply HRTF spatialization effect.

Filename	Description
`spatialize_param.m`	(function) Apply parametric HRTF spatialization effect.
`SpectralEntropy.m`	(function) Compute the spectral entropy feature from input signal.
`spectrogram.m`	(function) Compute the STFT of the input signal.
`start_compressor.m`	(deprecated script) Attempt to estimate dynamic range compression parameters from input signal.
`subsample_db2.m`	(function) Take a sub-sample of a given database corresponding to the given ratio of the entire database.
`summary_feat.m`	(function) Compute summary features from the input time series (mean, median, iqr, skewness, kurtosis, max, min, entropy, slope).
`summary_feat2.m`	(function) Compute summary features for the input matrix (*cf.* CD1-CD5 in [35]).
`test_comp.m`, `test_comp2.m`	(deprecated script) Dynamic range compression simulations.
`test_degradations.m`	(script) Signal alteration simulations.
`test_src_separation.m`	(deprecated script) Source separation simulation based on DUET method and Spatiali diversity.
`train_lda.m`	(function) Train LDA before classification (`pred_lda`).

5 Conclusion and future works

We have proposed a set of audio features for the automatic characterization of the audio quality. During an experiment with artificially generated mixing, we showed that the proposed approach can efficiently predict the type of audio effects and alterations applied to the original audio signal. With real commercial music tracks, we also showed that the same approach can be used to predict the decade during which the track was recorded. This approach paves the way of more sophisticated systems designed for automatic mixing, playlist generation or database indexing. Future works will consist in further investigating a larger set of "realistic" signal alterations and using audio quality annotated dataset as in [1] with further consideration to dynamic compression detection and modeling (model-based approach) and the development of new features (*e.g.* spatial diversity).

A Appendix

b	central freq. f_c [Hz]	$f_{\min}$ [Hz]	$f_{\max}$ [Hz]	Δ_f [Hz]
1	60	20	100	80
2	150	100	200	100
3	250	200	300	100
4	350	300	400	100
5	450	400	510	110
6	570	510	630	120
7	700	630	770	140
8	840	770	920	150
9	1000	920	1080	160
10	1170	1080	1270	190
11	1370	1270	1480	210
12	1600	1480	1720	240
13	1850	1720	2000	280
14	2150	2000	2320	320
15	2500	2320	2700	380
16	2900	2700	3150	450
17	3400	3150	3700	550
18	4000	3700	4400	700
19	4800	4400	5300	900
20	5800	5300	6400	1100
21	7000	6400	7700	1300
22	8500	7700	8500	1800
23	10500	8500	9500	2500
24	13500	9500	15500	3500

Table 14: Critical bands according to [25].

Profile name	sox parameters / commandline
speech 1	`compand 0.02,0.20 5:-60,-40,-10 -5 -90 0.1`
speech 2	`compand 0.3,5 6:-70,-60,-20 -10 -6 0.2`
speech/streaming	`compand 0.01,1 -90,-90,-70,-70,-60,-20,0,0 -5`
streaming 1	`compand 0.3,1 6:-70,-60,-20 -5 -90`
streaming 2	`compand 0.1,0.8 60:-40,-5 -20 -70 .2`
speech/streaming	`compand 0.01,1 -90,-90,-70,-70,-60,-20,0,0 -5`
speech/music 1	`compand 0.1,0.3 -60,-60,-30,-15,-20,-12,-4,-8,-2,-7 -2`
speech/music 2	`compand 0.0,1 6:-70,-43,-20 -6 -90 0.1`
music 1	`compand 0.3,1 -90,-90,-70,-70,-60,-20,0,0 -10 0 0.2`
music 2	`compand 0.3,1 6:-70,-60,-20 -5 -90 0.2`
music 3	`compand 0.3,1 -90,-90,-70,-70,-60,-20,0,0 -5 0 0.2`
music 4	`compand .1,.1 -60,-10 0 0 .1`
music 5	`compand .01,.3 -6,-4,-3,-3,0,-3`

Table 15: Commonly used sox dynamic range compression profiles as proposed in `http://forum.doom9.org/showthread.php?t=165807`.

References

[1] Alex Wilson and Bruno M Fazenda, "Perception of audio quality in productions of popular music," *Journal of the Audio Engineering Society*, vol. 64, no. 1/2, pp. 23–34, 2016.

[2] Alex Wilson and Bruno Fazenda, "Relationship between hedonic preference and audio quality in tests of music production quality," in *Proc. IEEE International Conference on Quality of Multimedia Experience (QoMEX)*, 2016, pp. 1–6.

[3] Paul Kendrick, Francis Li, Bruno Fazenda, Iain Jackson, and Trevor Cox, "Perceived audio quality of sounds degraded by nonlinear distortions and single-ended assessment using hasqi," *Journal of the Audio Engineering Society*, vol. 63, no. 9, pp. 698–712, 2015.

[4] Bruno Fazenda, Paul Kendrick, Trevor Cox, Francis Li, and Iain Jackson, "Perception and automated assessment of audio quality in user generated content," in *139th AES Convention*, Manchester, UK, Oct. 2015.

[5] Alex Wilson and Bruno M Fazenda, "A lexicon of audio quality," in *Proceedings of the 9th Triennial conference of the European Society for the Cognitive Sciences of Music (ESCOM 2015), Manchester, UK*, 2015.

[6] Pedro Duarte Pestana, Zheng Ma, Joshua D. Reiss, Alvaro Barbosa, and Dawn A. A. Black, "Spectral characteristics of popular commercial recordings 1950-2010," in *135th AES Convention*, NY, USA, Oct. 2013.

[7] D. Tardieu, E. Detruty, and G. Peeters, "Production effect: Audio features for recordings techniques description and decade prediction," in *Proc. Digital Audio Effects Conf. (DAFx'11)*, Sept. 2011, pp. 441–446.

[8] S. Gorlow and S. Marchand, "Reverse engineering stereo music recordings pursuing an informed two-stage approach," in *Proc. Digital Audio Effects Conf. (DAFx'13)*, 2013.

[9] O. Yilmaz and S. Rickard, "Blind separation of speech mixtures via time-frequency masking," *IEEE Transactions on Signal Processing*, vol. 52, no. 7, pp. 1830–1847, 2004.

[10] J. Ganseman, P. Scheunders, and S. Dixon, "Improving plca-based score-informed source separation with invertible constant-q transforms," in *Proc. European Signal Processing Conference (EUSIPCO'12)*, 2012, pp. 2634–2638.

[11] Y. Hioka, K. Furuya, K. Kobayashi, K. Niwa, and Y. Haneda, "Underdetermined sound source separation using power spectrum density estimated by combination of directivity gain," *IEEE Transactions on Audio, Speech, and Language Processing (TASLP)*, vol. 21, no. 6, pp. 1240–1250, 2013.

[12] K. Bronner and H. Rainer, *Audio Branding. Brands, Sound and Communication*, Nomos, 2009.

[13] D. Hughes and B. Manaris, "Fractal dimensions of music and automatic playlist generation: Similarity search via mp3 song uploads," in *Intelligent Information Hiding and Multimedia Signal Processing (IIH-MSP), 2012 Eighth International Conference on*, 2012, pp. 436–440.

[14] Daniele Barchiesi and Josh Reiss, "Automatic target mixing using least-squares optimization of gains and equalization settings," in *Proc. Digital Audio Effects Conf. (DAFx'09)*, 2009, pp. 7–14.

[15] Joerg Bitzer and Juergen Herre, "Coding of moving pictures and audio ISO/IEC JTC 1/SC 29/WG 11," Tech. Rep., International Organization for Standardization Organization Internationale Normalization, Shanghai, China, Oct. 2002.

[16] G. Peeters, B. Giordano, P. Susini, N. Misdariis, and S. McAdams, "The timbre toolbox: Audio descriptors of musical signals," *Journal of Acoustic Society of America (JASA)*, vol. 5, no. 130, pp. 2902–2916, Nov. 2011.

[17] G Widmer, K Seyerlehner, T Pohle, and M Schedl, "Automatic music detection in television productions," in *Proc. of the International Conference on Digital Audio Effects*, 2007.

[18] A. Wilson and B. Fazenda, "Characterisation of distortion profiles in relation to audio quality," in *Proc. Digital Audio Effects Conf. (DAFx'14)*, Sept. 2014.

[19] S. Gorlow and J. D. Reiss, "Model-based inversion of dynamic range compression," *IEEE Transactions on Audio, Speech, and Language Processing*, vol. 21, no. 7, pp. 1434–1444, 2013.

[20] J. Reiss D. Barchiesi, "Reverse engineering of a mix," *Journal of the Audio Engineering Society*, vol. 58, pp. 563–576, 2010.

[21] G.J. Brown and M.P. Cooke, "Computational auditory scene analysis," *Computer Speech and Language*, vol. 8, pp. 297–336, 1994.

[22] Carlos Avendano, "Frequency-domain source identification and manipulation in stereo mixes for enhancement, suppression and re-panning applications," in *Proc. IEEE WASPAA'03*, Sept. 2003, pp. 55–58.

[23] R.D. Patterson, K. Robinson, J. Holdsworth, D. McKeown, C. Zhang, and M. Allerhand, "Complex sounds and auditory images," *Auditory physiology and perception*, pp. 123–177, 1992.

[24] B.R. Glasberg and B.C.J. Moore, "Derivation of auditory filter shapes from notched-noise data," *Hearing Research*, vol. 47, Issues 1–2, pp. 103–138, 1990.

[25] E. Zwicker, "Subdivision of the audible frequency range into critical bands," *The Journal of the Acoustical Society of America*, vol. 33, no. Issue 2, pp. 248–248, 1961.

[26] A. Jourjine, S. Rickard, and O. Yilmaz, "Blind separation of disjoint orthogonal signals: demixing n sources from 2 mixtures," in *Proc. IEEE ICASSP'00*, 2000, vol. 5, pp. 2985–2988.

[27] D. Fourer, J-L. Rouas, P. Hanna, and M. Robine, "Automatic timbre classification of ethnomusicological audio recordings," in *Proc. ISMIR'14*, Taipei, Taiwan, Oct. 2014, pp. 295–300.

[28] R. Bittner, J. Salamon, M. Tierney, M. Mauch, C. Cannam, and J. P. Bello, "Medleydb: A multitrack dataset for annotation-intensive MIR research," in *Proc. ISMIR'14*, Taipei, Taiwan, Oct. 2014.

[29] T. W. Anderson, *An Introduction to Multivariate Statistical Analysis*, Wiley-Blackwell, NY, USA, 1958.

[30] F. Lauer and Y. Guermeur, "MSVMpack: a multi-class support vector machine package," *Journal of Machine Learning Research*, vol. 12, pp. 2269–2272, 2011, `http://www.loria.fr/~lauer/MSVMpack`.

[31] G. A. F. Seber, *Multivariate Observations*, Hoboken, NJ: John Wiley & Sons, Inc.

[32] G. Peeters, "Automatic classification of large musical instrument databases using hierarchical classifiers with inertia ratio maximization," in *115th AES Convention*, NY, USA, Oct. 2003.

[33] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*, Cambridge University Press., 2008.

[34] A. Flexer, "A closer look on artist filters for musical genre classification," in *Proc. ISMIR'07*, 2007, pp. 341–344.

[35] D. Fourer and G. Peeters, "Objective characterization of audio signal quality: applications to music collection description," in *Proc. IEEE ICASSP'2017*, 2017, p. submitted.